Lecture 07: Network Basics

Objectives:

- Learn how network communication is achieved using protocols & Layers
- Learn the TCP/IP Protocol Stack : TCP, UDP, IP

1. Protocols and Layers

If we have a large problem to be solved and coded using Java, we normally break the solution into modules and create a number of classes, where each class have a distinct purpose, with defined functionality.

This approach reduces the complexity of the problem into a number of smaller chunks, each of which partially contributes to the solution of the larger problem.

A similar process is followed in networking. For example, sending an E-mail from one computer to another (which might be on the other side of the world) is a very large and complex task. In order to solve this task we break the problem into a number of smaller problems. Each smaller problem is then solved; not through the creation of a Java class, but instead through the creation of a ***protocol***.

Protocols provide a common set of rules which computers on the network can use to communicate with one another. They specify how information is moved around the network, how computers should behave when connected to the network, etc. Throughout this course we will encounter many different protocols.

The following example illustrates how a large task can be broken down into a number of smaller tasks, with each task 'solved' through the use of an appropriate protocol.

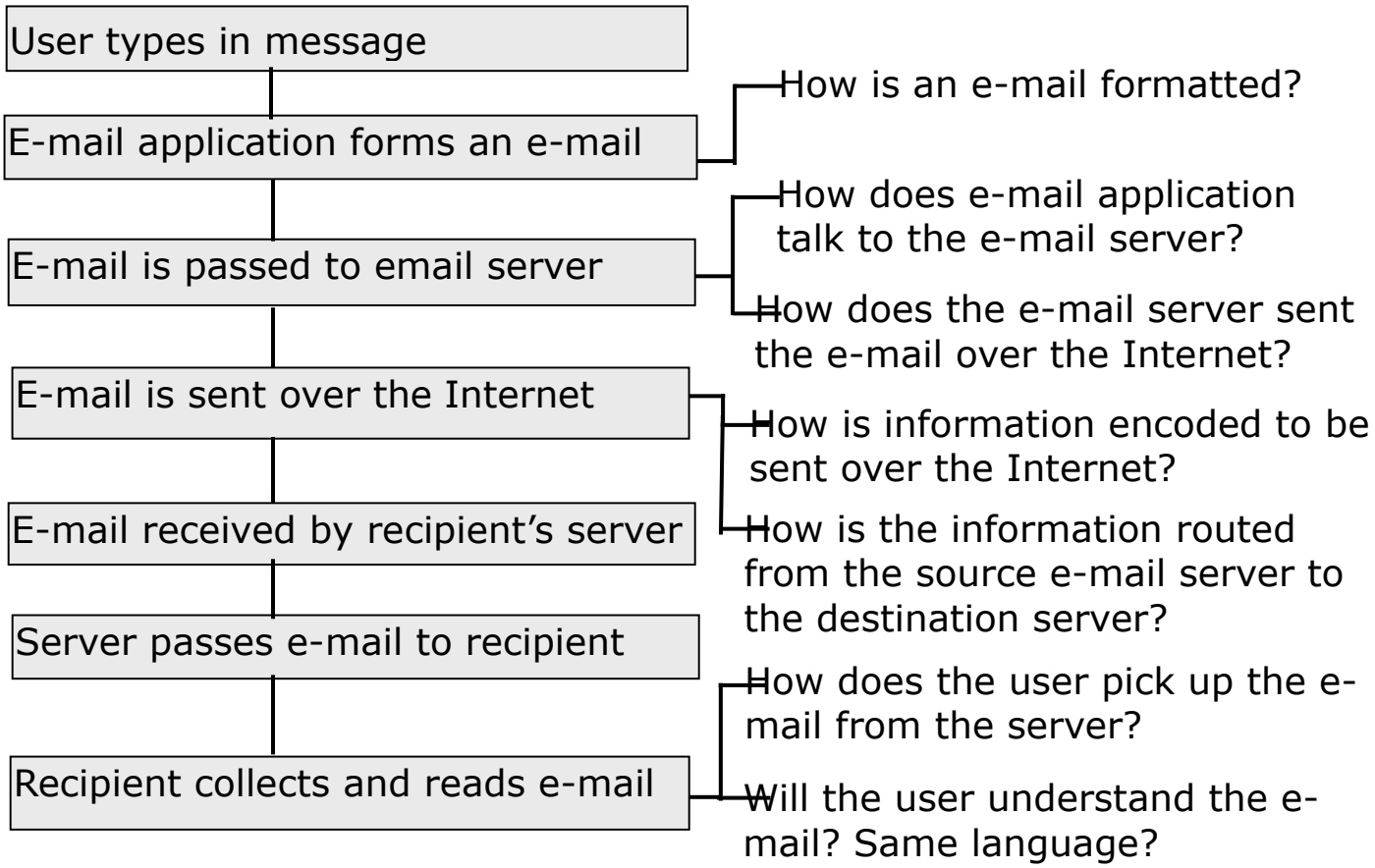
Example 1: Email Sending Problem:

Whenever you send an E-mail, your E-mail client (e.g. Outlook, etc.) takes the message, turns it into an E-mail and finally sends it to your E-mail server.

The E-mail server passes the message onto the 'Internet', over which the E-mail travels to the correct destination.

The receiving E-mail server keeps the message until the recipient checks to see if they have any new mail. At this point the recipient's E-mail client displays the E-mail.

Some of the issues involved in this process are identified below:



Given the above description, we can break the process up into a number of stages, with each stage having its own particular set of problems.

For example, we can identify an E-mail client, an E-mail server and our Internet connection.

The E-mail client is responsible for formatting and displaying the E-mail, the server for sending and receiving E-mails, and the Internet for the actual delivery of the E-mails.

In terms of networking terminology, we have broken the larger problem up

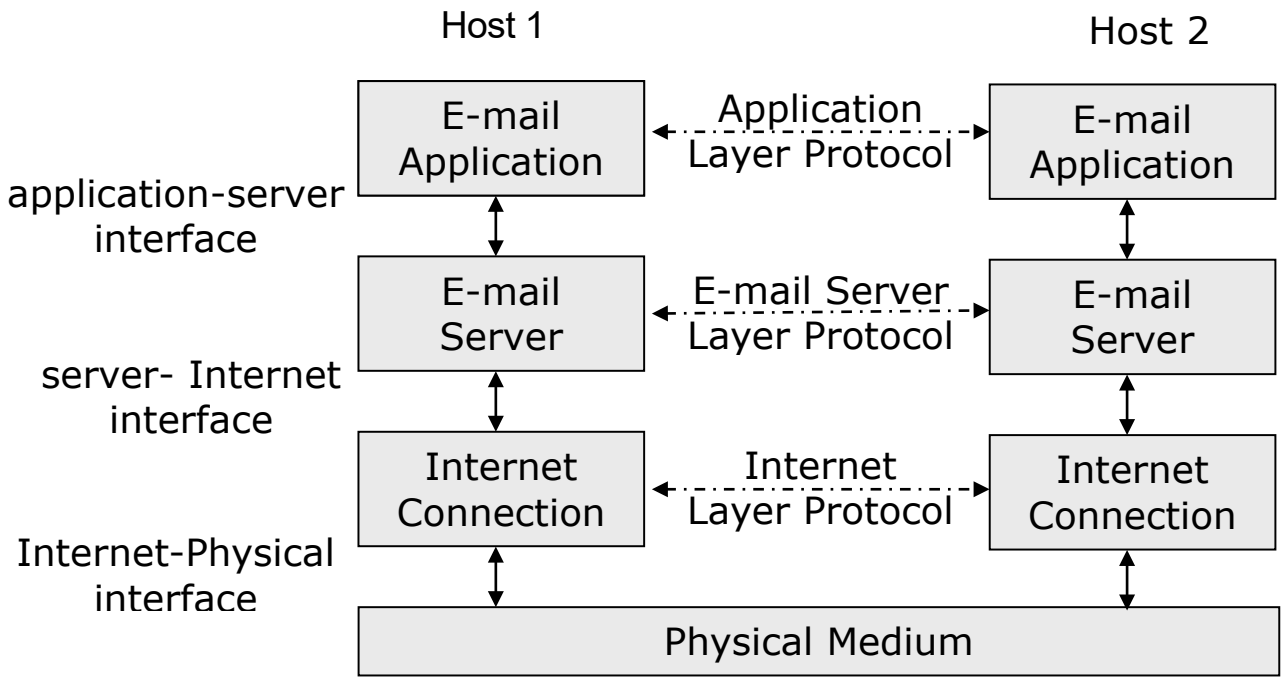
into a number of ***layers***, with each layer performing a well defined function.

Each layer can talk to other layers through the use of an ***interface***.

Finally, we can determine how each layer will solve its own particular set of problems. For example, we can define how E-mails will be formatted, how images are to be displayed, etc.

In terms of networking terminology we define the ***protocol*** to be used within this layer.

One example of a layered solution to our E-mail problem is shown below:



The layered approach views the network as consisting of a number of layers which communicate with one another through the use of interfaces.

Each layer offers certain services to the higher layers, whilst shielding from those layers the details of how the offered services are actually implemented.

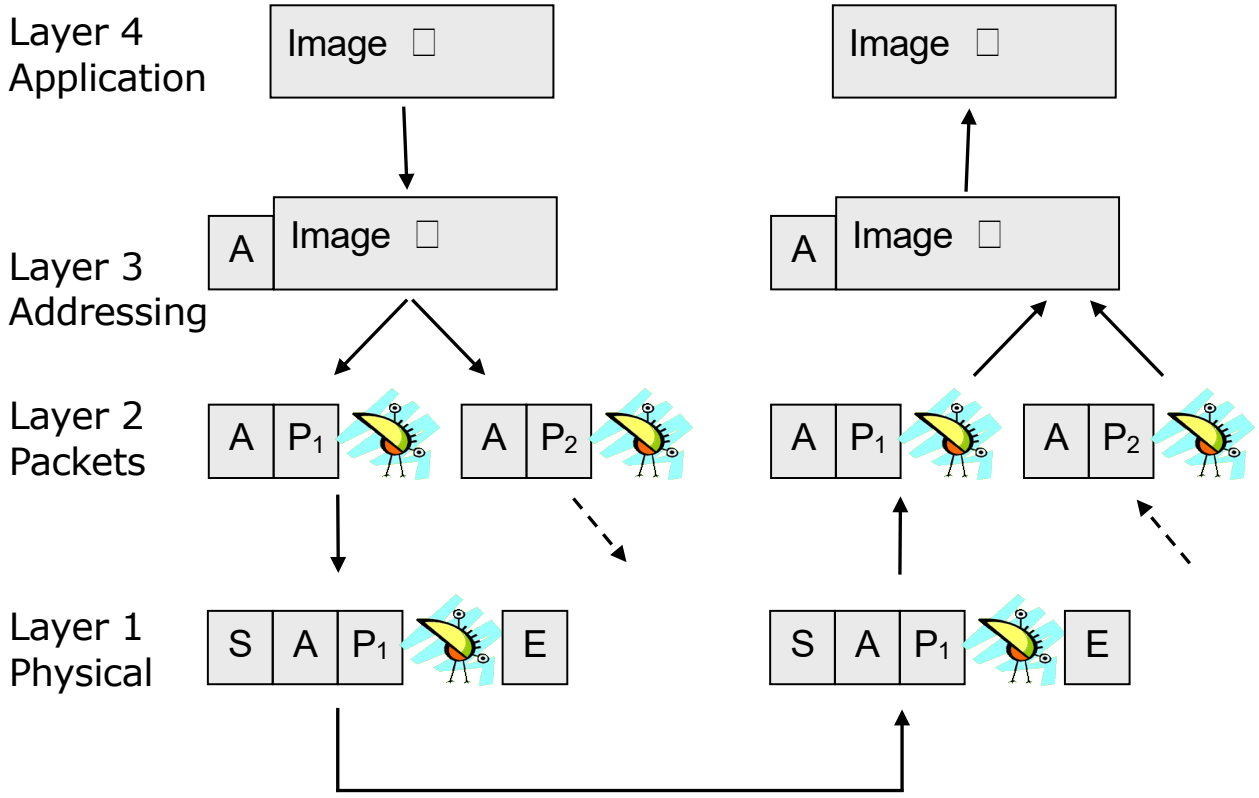
In general, lower level layers take the data provided by higher-level layers and transforms it into a format more suitable to being sent over the

network (ultimately some form of physical medium).

A set of layers and protocols is termed ***network architecture***.

Example 2: Layered communication

An example of how a layered approach can be used when sending an image from one computer to another follows:



- The application running on layer 4 wishes to send an image to another computer. In order to do this it passes the image data to layer 3.
- Layer 3 is responsible for addressing, taking the image data and appending addressing information that uniquely identifies the recipient. The message is then passed to layer 2.
- Layer 2 takes the data and splits it up into smaller pieces, called ***packets***, which are suitable for being sent over the network. Indexing information is added to the sequence of packets so that the data can be reconstructed in the correct order when it reaches the recipient. Each packet is passed to layer 1.
- Layer 1 is responsible for actually sending packets over the network to the intended recipient. In order to do this successfully it adds transmission information (e.g. packet start and end identifiers, error

correction codes, etc.). Finally the layer 1 process sends the data over the physical medium to the intended recipient.

The reverse process occurs at the recipient's computer, e.g. packets are collected and checked for errors, before being assembled back into the original sequence. Finally the reconstructed image is passed to the application layer.

The outlined process of peer abstraction (e.g. layer N on one computer talks directly to layer N on another computer) is fundamental to most network designs.

Using this approach, the difficult task of designing a complete network solution can be broken into several smaller, more manageable design problems (namely the design of individual layers) which are separated from one another through the layer interfaces.

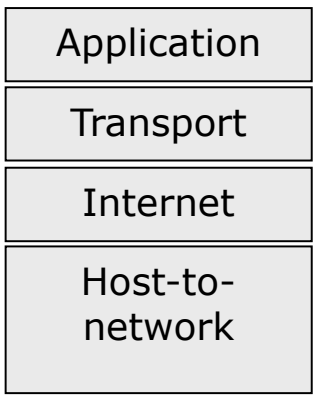
The set of layers together with their associated protocols is called a ***protocol stack***.

2. TCP/IP Protocol Stack

As with virtually all other networks, the operation of the Internet can be described in terms of a number of layers and protocols.

In this course we are interested in understanding the network architecture that permits a range of different applications to communication with one another over the Internet.

The particular collection of layers and protocols that describes the operation of the Internet is known as the TCP/IP Protocol Stack, also known as TCP/IP Reference Model.



At its highest level the TCP/IP model is as follows:

TCP/IP model operates using a total of four layers.

The Application layer contains all the useful application protocols.

The Transport layer is concerned with how to transfer

information from applications onto the Internet.

The Internet layer is concerned with determining how information can get from point A to point B.

Finally, the Host-to-Network layer is responsible for the actual transfer of data across the physical network.

Each layer is further explained below.

The Application Layer

This is the highest-level layer in the TCP/IP model, and sits on top of the transport layer.

It contains all of the higher-level protocols (many of which will be encountered in this course). Examples are:

- TELNET – A virtual terminal (i.e. offering login to distance machines)
- FTP – File transfer (permitting files to be efficiently transferred from one machine to another).
- SMTP – Electronic mail
- NNTP – Network News Transfer protocol,
- HTTP, etc.

The Transport Layer

The goal of the Transport Layer is to manage the communication process between two different hosts.

Manage includes the process of establishing connections, transferring and validating data, closing connections, etc.

Two different end-to-end protocols have been defined:

- ***TCP**: **Transmission Control Protocol*****:** a reliable protocol that permits a byte stream to be delivered without error to any other machine on the Internet. It operates by fragmenting the byte stream into discrete packets that are then passed onto the internet layer.

  TCP protocol ensures that the packets are reordered correctly; that corrupted or missing packets are retransmitted; that a fast sender does

not swamp a slow recipient, etc. The expense of this added functionality is a reduction in speed.

- ***UDP**: **User Datagram Protocol*:** an unreliable protocol for applications that do not wish to use TCP's flow control. Useful if it does not really matter if some packets are lost or received in the wrong order. It is widely used within applications for which prompt delivery is more important than accurate delivery, such as the transmission of images, audio or video.

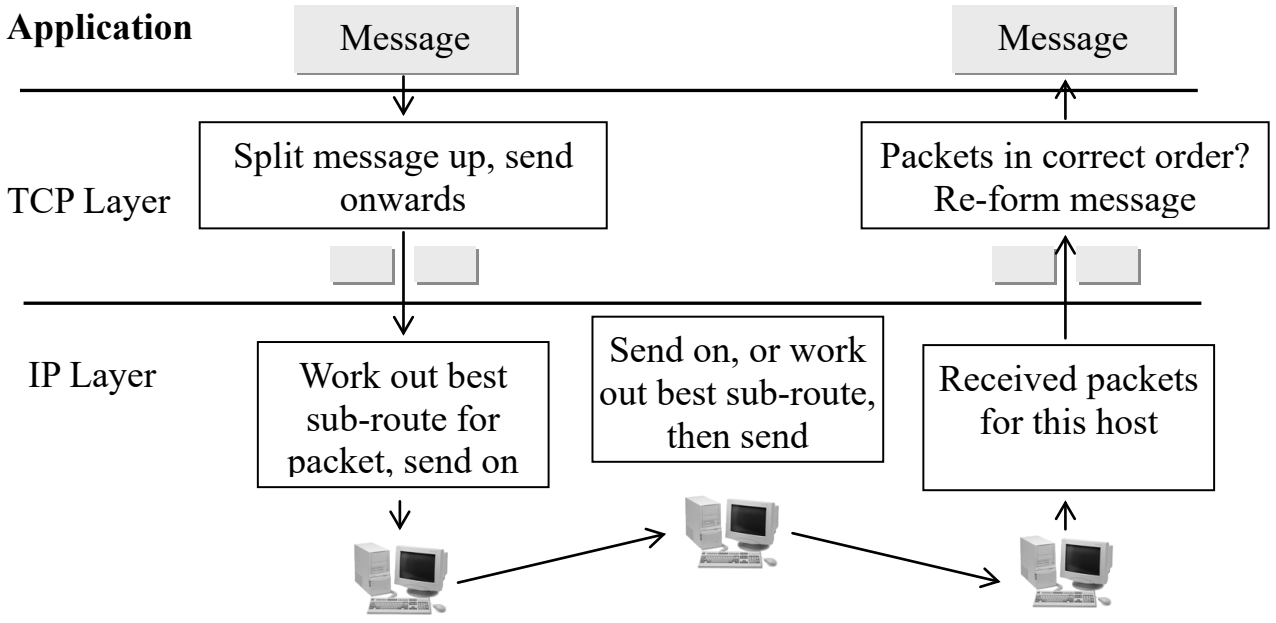
Note that these two protocols are defined within the same layer (i.e. TCP and UDP). Each protocol is capable of satisfying the 'functionality' required of the layer, i.e. managing a communication process.

The Internet Layer (also called IP layer)

The internet layer (to be differentiated from 'The Internet') provides the means through which a host can send packets onto a network and then have them travel independently to their destination (possibly on another network on the other side of the world).

This role gives the layer its name, i.e. it is in effect an internet, connecting different networks together. This layer routes each packet individually, thus, packets may arrive in a different order to which they were sent.

The internet layer defines an official packet format and protocol called IP (Internet Protocol). It is the job of the IP protocol to deliver the IP packets to their destination; hence packet routing is primarily the main purpose of this layer.



The Host-to-Network Layer

This is mostly undefined within the TCP/IP model. The reason is that the TCP/IP model was designed to work on top of a wide range of different network types.

Hence, the TCP/IP reference model simply states that the IP layer must be able to connect to the underlying network using some (unspecified protocol) so that it can send IP packets over the network.

The actual protocol that is used depends upon the underlying network (e.g. Ethernet, etc.).

As the popularity of the TCP/IP model increased most network manufacturers provided a protocol to support the IP layer.

TCP/IP Reference Model Summary

A summary of the TCP/IP reference model follows:

